

Министерство цифрового развития, связи и массовых коммуникаций РФ
федеральное государственное бюджетное образовательное учреждение высшего
образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

Кафедра вычислительных систем
Допустить к защите
Зав. кафедрой к.т.н., доцент
_____ Перышкова Е.Н.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

Разработка программы «Голосовой ассистент» для
Windows

Пояснительная записка

Студент Алексеев К.Н.

Институт ИВТ Группа ИВ-022

Руководитель д.т.н. доцент Мамоиленко С.Н.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 АНАЛИЗ АРХИТЕКТУРЫ ПРОГРАММЫ	6
2 СРЕДСТВА РАЗРАБОТКИ.....	7
2.1 Язык программирования C#.....	7
2.2 Движок распознавания активационного слова Picovoice porcupine	7
2.3 Библиотека распознавания речи VOSK	8
2.4 Облачный сервис Yandex SpeechKit.....	8
2.5 Библиотека работы со звуком NAudio	8
2.6 Обработка естественного языка на основе правил.....	9
2.7 Библиотека FuzzySharp.....	9
3 ПРОЕКТИРОВАНИЕ.....	11
3.1 Структура программы.....	11
3.1.1 Модуль распознавания речи.....	11
3.1.2 Структура команды	11
3.1.3 Модуль обработки естественного языка.....	12
3.1.4 Модуль синтеза речи.....	12
3.2 Взаимосвязь компонентов	12
4 РАЗРАБОТКА МОДУЛЯ РАСПОЗНАВАНИЯ РЕЧИ	14
4.1 Библиотека VOSK	14
4.2 Обучение языковой модели	15
4.3 Разработка класса Recognizer на C#.....	15
5 РАЗРАБОТКА КОМАНД И ИХ СТРУКТУРА.....	18
5.1 Команда «Приветствие»	18
5.2 Команда «Запуск»	19
5.3 Команда «Управление звуком»	20
5.3.1 Вспомогательный класс AudioManager	20
5.3.2 Основной класс SoundControlCommand	21
5.3.3 Вспомогательный класс StringToNumberConverter	22

5.4 Команда «Погода».....	23
5.4.1 Вспомогательный класс WeatherService	23
5.4.2 Основной класс WeatherCommand	24
5.5 Команда «Поиск в википедии».....	25
5.6 Алгоритм мягкого сравнения.....	27
6 РАЗРАБОТКА МОДУЛЯ «МЕНЕДЖЕР КОМАНД»	28
7 РАЗРАБОТКА МОДУЛЯ СИНТЕЗА РЕЧИ.....	32
ЗАКЛЮЧЕНИЕ	36
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ.....	38
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	39
ПРИЛОЖЕНИЕ А	41
ПРИЛОЖЕНИЕ Б.....	43
ПРИЛОЖЕНИЕ В.....	46
ПРИЛОЖЕНИЕ Г	48
ПРИЛОЖЕНИЕ Д	50
ПРИЛОЖЕНИЕ Е.....	52
ПРИЛОЖЕНИЕ Ж.....	54

ВВЕДЕНИЕ

Развитие технологий искусственного интеллекта и машинного обучения повлияло на широкий спектр областей, включая взаимодействие человека с компьютером. Одной из ключевых технологий, активно развивающихся в последние годы, является голосовой ассистент. Siri, Google Assistant и Alexa, интегрируются в повседневную жизнь людей, упрощая доступ к информации и управлению устройствами. В то время как голосовые ассистенты широко распространены на мобильных платформах и умных колонках, их использование на десктопных операционных системах, таких как Windows, все еще находится на начальной стадии развития.

Актуальность данной работы обусловлена растущим спросом на более интуитивные и естественные способы взаимодействия пользователей с компьютерами. Несмотря на существование таких решений, как Cortana (на данный момент поддержка прекращена), многие пользователи и разработчики отмечают ограниченные возможности и необходимость в более гибких и функциональных системах. Это создает предпосылки для разработки нового голосового ассистента.

Исходные данные для разработки данной темы включают в себя современное состояние технологий распознавания и синтеза речи, прогресс в области обработки естественного языка (NLP) и имеющиеся наработки в создании голосовых интерфейсов. В последние годы наблюдается значительный прогресс в этих областях, что позволяет создавать более точные и функциональные голосовые ассистенты.

Необходимость проведения данной работы обосновывается потребностью в создании голосового ассистента, который бы соответствовал современным требованиям пользователей, обеспечивал высокий уровень точности распознавания и синтеза речи, быструю обработку и выполнение пользовательских команд.

Целью данного проекта является разработка функционального голосового ассистента, который сможет обрабатывать и выполнять команды

пользователя, используя технологии распознавания и синтеза речи. Для достижения этой цели необходимо выполнить следующие задачи:

1) Исследование и анализ существующих решений:

- Анализ существующих голосовых ассистентов и их функционала.
- Изучение технологий и библиотек для распознавания и синтеза речи, доступных для платформы Windows.

2) Разработка архитектуры программы:

- Определение основных компонентов системы.
- Разработка схемы взаимодействия между компонентами.

3) Реализация функциональности голосового управления:

- Интеграция технологий распознавания речи для обработки голосовых команд.
- Разработка модуля «Менеджер команд» для обработки естественного языка
- Реализация функционала синтеза речи для обратной связи с пользователем.

1 АНАЛИЗ АРХИТЕКТУРЫ ПРОГРАММЫ

При разработке голосового ассистента разработчик сталкивается с тремя основными проблемами:

Задача распознавания речи заключается в преобразовании звуковых сигналов речи в текст. Система должна точно интерпретировать произнесенные слова, несмотря на различия в акцентах, произношении и условиях записи (например, наличие фонового шума);

После того как речь преобразована в текст, необходимо понять, что именно хотел сказать пользователь. Это включает в себя анализ текста для выявления намерений, понимание контекста диалога, а также генерацию корректных ответов. Системы NLP должны уметь обрабатывать широкий спектр выражений, включая идиомы, сленг и неточности в речи, чтобы обеспечивать естественное и эффективное взаимодействие с пользователем.

После того как система определила, что ответить пользователю, необходимо преобразовать этот текст обратно в речь. Этот этап требует создания естественного и приятного звучания, которое будет легко восприниматься пользователем. Система TTS должна учитывать интонацию, ритм и ударения, чтобы синтезированная речь звучала как можно более приближенно к человеческой. Кроме того, синтез речи должен происходить быстро, чтобы взаимодействие было плавным и непрерывным.

Подводя итог можно выделить три основных модуля программы:

- 1) Модуль синтеза речи;
- 2) Модуль обработки естественного языка;
- 3) Модуль распознавания речи.

2 СРЕДСТВА РАЗРАБОТКИ

2.1 Язык программирования C#

C# был выбран в качестве основного языка программирования. Это обусловлено несколькими причинами:

- 1) интеграция с Windows;
- 2) высокая степень контроля над кодом благодаря строгой типизации;
- 3) богатая стандартная библиотека для разработки различных компонентов,
- 4) активное сообщество и обширная документация, что облегчает процесс разработки и поиска решений возникающих проблем.

Аналогами C# являются языки программирования Python и Java. Python популярен благодаря своей простоте и богатой экосистеме библиотек, особенно для научных и аналитических задач, включая машинное обучение. Java, как и C#, обладает строгой типизацией и используется для разработки масштабируемых корпоративных приложений. Однако C# предпочтителен для разработки под Windows из-за тесной интеграции с .NET Framework и .NET Core.

2.2 Движок распознавания активационного слова Picovoice porcupine

Picovoice был выбран для обнаружения активационного слова. Преимущества Picovoice:

- 1) высокую точность распознавания активационных слов даже в шумных условиях;
- 2) возможность работы офлайн, что обеспечивает приватность и быстроту отклика;
- 3) низкое потребление ресурсов.

Аналогами Picovoice являются Google Voice Recognition и Snowboy. Google Voice Recognition обеспечивает высокую точность и интеграцию с Google-сервисами, но требует постоянного интернет-соединения. Snowboy

работает офлайн и имеет хорошую точность, но Picovoice эффективнее в шумных условиях и потребляет меньше ресурсов.

2.3 Библиотека распознавания речи VOSK

Для распознавания речи был использован VOSK. Этот инструмент имеет открытый исходный код, что позволяет модифицировать его под конкретные нужды и использовать без лицензионных ограничений. VOSK поддерживает множество языков, включая русский, и может работать без подключения к интернету. Аналогами VOSK являются Kaldi и Yandex STT. Kaldi также обладает открытым исходным кодом и высокой точностью, но требует более сложной настройки. Yandex STT использует современные нейронные сети для распознавания речи и обеспечивает высокую точность, но для его работы часто требуется постоянное подключение к интернету.

2.4 Облачный сервис Yandex SpeechKit

Для синтеза речи был выбран Yandex SpeechKit, который обеспечивает высокое качество синтезированной речи и поддержку русского языка, что важно для локализации ассистента. Так как синтез речи будет использоваться в командах, требующих доступ к интернету (например, прогноз погоды или ответы от Yandex GPT), было решено использовать облачный сервис в угоду качеству синтезируемого текста. Аналогами Yandex SpeechKit являются Google Text-to-Speech и Amazon Polly. Google Text-to-Speech, но Yandex SpeechKit предпочтителен для русского языка и интеграции с российскими сервисами.

2.5 Библиотека работы со звуком NAudio

Для работы со звуком был выбран NAudio. Этот инструмент предоставляет широкий набор функций для записи, воспроизведения, обработки и анализа аудио данных. NAudio легко интегрируется с .NET-приложениями, что делает его идеальным выбором для использования с C#. Кроме того, NAudio активно развивается и поддерживается сообществом, что обеспечивает наличие обновлений и исправлений. Аналогами NAudio

являются BASS и FMOD. Они предоставляют широкий функционал для работы со звуком и имеют высокую производительность, но требуют коммерческой лицензии для некоторых видов использования.

2.6 Обработка естественного языка на основе правил

Для обработки естественного языка (NLP) в проекте был использован подход на основе правил (Rule-based). Этот метод позволяет создавать набор четких и предсказуемых правил для анализа и обработки пользовательских команд. Rule-based подход был выбран из-за его простоты и эффективности в условиях, когда требуется обработка структурированных запросов. Такой подход обеспечивает высокий уровень контроля над процессом обработки команд и позволяет быстро вносить изменения и улучшения в систему, что важно на этапе разработки и тестирования ассистента. Аналогами Rule-based подхода являются методы, основанные на машинном обучении, такие как использование нейронных сетей и трансформеров. Методы машинного обучения могут обрабатывать более сложные и вариативные запросы, но требуют больших объемов данных для обучения и более мощного оборудования. В отличие от них, Rule-based подход проще в реализации и не требует значительных вычислительных ресурсов.

2.7 Библиотека FuzzySharp

FuzzySharp — это библиотека для C#, предназначенная для выполнения нечеткого поиска и сравнения строк. Она позволяет определять степень схожести между строками, даже если они не совпадают точно. Это особенно полезно в случаях, когда необходимо сопоставить пользовательский ввод с заранее определенными значениями, но ввод может содержать опечатки или небольшие отклонения.

Библиотека использует различные алгоритмы для оценки схожести строк, такие как алгоритмы Левенштейна и Дамерау-Левенштейна, которые вычисляют минимальное количество операций (вставка, удаление или замена символов), необходимых для превращения одной строки в другую. Эти

алгоритмы позволяют оценить, насколько сильно две строки отличаются друг от друга.

FuzzySharp также включает алгоритмы, такие как Jaccard и Sorensen-Dice, которые основываются на теории множеств и вычисляют коэффициенты схожести, сравнивая подмножества символов или биграммы в строках. Ещё один популярный метод — алгоритм Джаро-Винклера, который учитывает как общее количество совпадающих символов, так и их относительное положение в строках, что делает его эффективным для сравнения коротких строк или строк с незначительными опечатками.

FuzzySharp предлагает широкий выбор алгоритмов для оценки схожести строк, таких как алгоритмы Левенштейна, Дамерау-Левенштейна, Джаро-Винклера, Jaccard и Sorensen-Dice. Это позволяет пользователям выбрать наиболее подходящий метод для их конкретных нужд, обеспечивая более точные результаты в разных сценариях.

Так же он позволяет разработчикам настраивать пороговые значения схожести, что дает возможность гибко управлять точностью и чувствительностью поиска. Это особенно полезно для создания более адаптивных и пользовательских решений, где требования к точности могут варьироваться в зависимости от контекста.

3 ПРОЕКТИРОВАНИЕ

3.1 Структура программы

3.1.1 Модуль распознавания речи

В модуле распознавания речи этого проекта применяется двухэтапная схема. На первом этапе система ищет активационное слово во входном аудиопотоке. Это слово служит сигналом для начала прослушивания пользовательских команд. По сути, это имя голосового ассистента, и его обнаружение позволяет программе активироваться только при непосредственном обращении пользователя.

На втором этапе происходит преобразование речи, записываемой с микрофона, в текстовый формат. Этот процесс обеспечивает интерпретацию команд пользователя. Таким образом, система работает эффективно, переводя речь в понятные для программы текстовые команды только при условии предварительного обращения пользователя к ассистенту.

3.1.2 Структура команды

Команда — это определенное действие или задача, которую пользователь может передать ассистенту, используя голосовую команду. Каждая команда обычно связана с определенной функциональностью или операцией, которую ассистент может выполнить. Например, запуск приложения, выполнение поиска в интернете, запуск сценария и т. д.

Каждая команда имеет ключевое слово и параметры, необходимые для выполнения этой команды. Параметр может быть один или несколько. Наглядно структура команды представлена на рисунке 3.1.

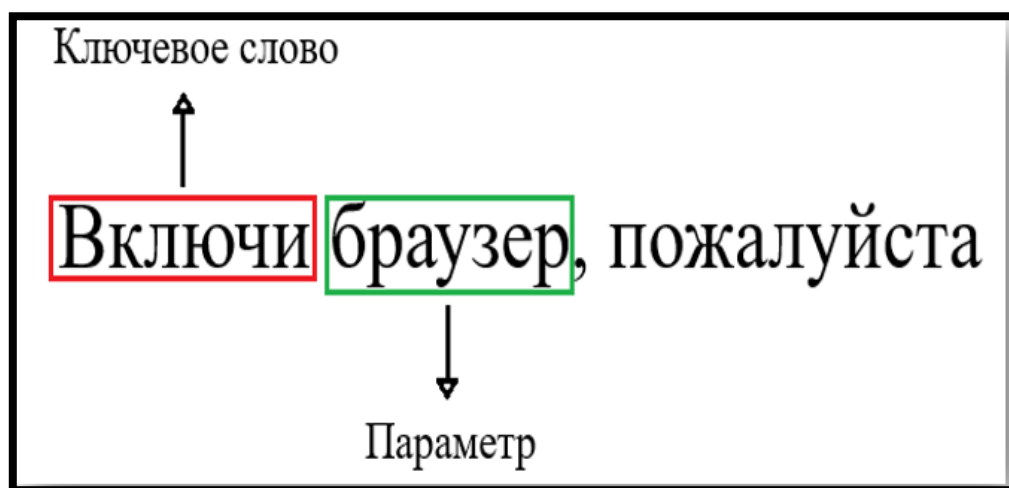


Рисунок 3.1 Структура команды

3.1.3 Модуль обработки естественного языка

Этот модуль сопоставляет текст, распознанный в первом модуле, с имеющимися командами и, если такая команда имеется, запускает ее выполнение. Вот его основной функционал:

- 1) Инициализация команд;
- 2) Поиск команды по ключевому слову;
- 3) Получение параметров команды.

3.1.4 Модуль синтеза речи

Этот модуль отвечает за синтез речи в голосовом ассистенте. Вот его функциональность:

Преобразование текста в аудиофайл формата MP3. Полученный аудиофайл возвращается в виде массива байт.

Воспроизведение синтезированной речи. Методы позволяют воспроизводить синтезированную речь. Один из методов принимает массив байт с аудиоданными и проигрывает их. Другой метод принимает путь к заранее записанному аудиофайлу и воспроизводит его.

3.2 Взаимосвязь компонентов

Взаимодействие между модулями распознавания речи, обработки естественного языка и синтеза речи в проекте выглядит следующим образом:

1) Активация и распознавание речи:

- Модуль распознавания речи постоянно прослушивает входящий аудиопоток.
- При обнаружении активационного слова система активируется и начинает записывать последующую речь пользователя.
- Записанная речь преобразуется в текст и передается в модуль обработки естественного языка.

2) Обработка текста:

- Модуль обработки естественного языка получает текст от модуля распознавания речи.
- Этот модуль анализирует текст для поиска ключевого слова, соответствующего одной из команд.
- После определения команды модуль извлекает необходимые параметры из текста.

3) Выполнение команды:

- Модуль обработки естественного языка передает команду и параметры модулям команд для выполнения.
- Если команда требует синтезированной речи, текст команды передается в модуль синтеза речи.

4) Синтез и воспроизведение речи:

- Модуль синтеза речи преобразует полученный текст в аудиофайл.
- Полученный аудиофайл воспроизводится, озвучивая ответ пользователю.

4 РАЗРАБОТКА МОДУЛЯ РАСПОЗНАВАНИЯ РЕЧИ

4.1 Библиотека VOSK

Для распознавания речи используется VOSK. VOSK – это инструмент с открытым исходным кодом для распознавания речи, который обеспечивает высокую точность и производительность в оффлайн-режиме. Он поддерживает множество языков, включая русский, и способен работать на различных платформах, таких как Windows, Linux, macOS и Android.

VOSK состоит из нескольких ключевых компонентов, которые обеспечивают его функциональность:

1) Акустические модели:

- Акустические модели VOSK обучены на большом количестве аудиоданных для распознавания звуков речи.
- Могут быть специфичными для различных языков и акцентов.

2) Языковые модели:

- Языковые модели помогают VOSK понимать контекст и предсказывать наиболее вероятные последовательности слов.
- Это повышает точность распознавания речи, особенно в сложных фразах и предложениях.

3) Декодер:

- Декодер объединяет акустические и языковые модели для преобразования аудио в текст.
- Он анализирует аудиопоток в реальном времени и возвращает текстовые данные.

4) Интерфейсы API:

- VOSK предоставляет интерфейсы API для различных языков программирования, включая Python, Java, C#, C++ и другие.

4.2 Обучение языковой модели

В проекте использовалась языковая модель `vosk-small-ru` которая является частью набора инструментов для распознавания речи `Vosk`, разработанного `Kaldi`. Она предназначена для выполнения задач по преобразованию речи в текст (`Speech-to-Text`, `STT`) и представляет собой компактную модель для русского языка.

Модель `vosk-small-ru` специально создана для работы на устройствах с ограниченными ресурсами, потребляя меньше памяти и вычислительных мощностей по сравнению с более крупными моделями. Благодаря обучению на большом количестве русскоязычных данных она обеспечивает высокую точность распознавания русской речи.

При разработке модуля распознавания речи возникла проблема: модель не распознавала названия многих программ, таких как `Steam`, `Paint`, `Visual Studio` и других. Для решения этой проблемы была заменена языковая модель, отвечающая за словарный запас. Выполнены следующие шаги:

- 1) Установлена система распознавания речи `Kaldi`.
- 2) Установлены дополнительные утилиты `IrstLM` и `OpenGRM`.
- 3) Установлена Python-библиотека `Phonetisaurus`.
- 4) Инициализированы пути к инструментарию.
- 5) В файл были добавлены слова, которые модель должна "запомнить".

Эти слова были включены в словосочетания, каждое на новой строке.

- 6) Запущен Python-скрипт, который сгенерировал модифицированную модель.

4.3 Разработка класса `Recognizer` на C#

`Recognizer` представляет собой класс реализующий функции распознавания речи и поиска активационного слова. На листинге 4.1 представлен конструктор класса `Recognizer`.

Листинг 4.1 - конструктор класса Recognizer

```
public Recognizer()  
{  
    porcupine = Porcupine.FromKeywordPaths(  
        new List<string> {PicoWWDModelPath},  
        modelPath: PicoModelPath);  
    model = new Model(VoskModelPath);  
    rec = new VoskRecognizer(model, SampleRate);  
    rec.SetMaxAlternatives(0);  
    rec.SetWords(false);  
}
```

В конструкторе данного класса инициализируется модель VOSK и движок Porcupine, а также устанавливаются параметры такие как количество альтернатив при распознавании и частота дискретизации.

Данный код (листинг A.1.) представляет собой асинхронный метод RecognizeAsync(), который используется для распознавания речи с помощью модели VOSK. Первая строка метода содержит задержку в 0.1 секунды, чтобы дать некоторое время для начала записи звука перед его обработкой. Это необходимо так как перед началом распознавания воспроизводится ответ от голосового ассистента и, если задержки не будет, он может записать это в фрагмент для распознавания.

Затем инициализируются переменные, в том числе строка result, которая содержит JSON-строку с пустым текстом. Создается объект WaveInEvent для захвата аудиоданных с микрофона, и устанавливается формат аудиофайла. Также создается TaskCompletionSource<bool>, который будет использоваться для уведомления о завершении записи. Добавляются обработчики событий DataAvailable и RecordingStopped. Первый вызывается при наличии новых аудиоданных и выполняет распознавание речи. Если распознавание завершено, результат записывается в переменную result, и запись звука останавливается. Второй обработчик вызывается после завершения записи и

устанавливает результат для выхода из асинхронной функции. После этого результат распознавания парсится из формата JSON, и возвращается текст распознанной речи.

Данный код (листинг A.2.) представляет собой асинхронный метод, который используется для распознавания активационного (wake word) с использованием библиотеки Picovoice.

Метод начинается с инициализации объекта `PvRecorder`, который представляет собой записывающее устройство для захвата аудиоданных. Далее метод запускает запись аудио с помощью метода `Start()`.

Затем выполняется бесконечный цикл, который продолжает выполнение до тех пор, пока запись продолжается (переменная `recorder.IsRecording` истинна). Внутри цикла создается асинхронная задача для чтения аудиофрейма с помощью метода `Task.Run(() => recorder.Read())`, чтобы выполнить чтение аудиофрейма параллельно с основным потоком выполнения.

Далее используется ключевое слово `await`, чтобы ожидать завершения асинхронной задачи чтения аудиофрейма и получить результат - массив аудиоданных типа `short[]`. Этот массив передается в метод `porcupine.Process(frame)` для обработки, и результат обработки сохраняется в переменной `result`.

Затем происходит проверка результата обработки. Если результат больше или равен нулю, что означает обнаружение активационного слова, запись останавливается с помощью метода `recorder.Stop()`, и метод возвращает `true`. Если слово не обнаружено в текущем аудиофрейме, цикл продолжается. По завершении записи метод возвращает `false`. Этот метод позволяет асинхронно обнаруживать "будильное слово" в потоке аудиоданных и возвращать `true`, если оно было обнаружено, или `false`, если нет.

5 РАЗРАБОТКА КОМАНД И ИХ СТРУКТУРА

Каждая команда в данном проекте представлена в виде класса, реализующего интерфейс ICommand. Его реализация приведена в листинге 5.1.

Листинг 5.1 - Интерфейс ICommand()

```
public interface ICommand
{
    public string[] TargetWords { get; }
    public Tuple<string, bool> Execute(List<string>
commandParams);
}
```

Данный интерфейс гарантирует, что у каждой команды будет перечень ключевых слов и метод Execute(), принимающий список параметров команды и отвечающий за ее выполнение. Метод возвращает пару string – bool. Если вернул true, то в строке лежит текст который нужно озвучить, если false, значит ответ уже записан и в строке лежит путь до файла, которой нужно воспроизвести.

5.1 Команда «Приветствие»

Данная команда (листинг 5.2) обрабатывает приветствие пользователя.

Листинг 5.2 – Класс HelloCommand

```
public class HelloCommand : ICommand
{
    public string[] TargetWords { get; private set; }
    public string[] PathsToAnswer { get; private set; }
    public HelloCommand()
    {
        TargetWords = ["привет", "здравствуй", "добрый день"];
        PathsToAnswer = ["HelloV1.mp3", "HelloV2.mp3", "HelloV3.mp3"];
    }
    public Tuple<string, bool> Execute(List<string> commaandParams)
    {
        bool textAnswer = false;
        Random rnd = new Random();
        return new Tuple<string, bool>(PathsToAnswer[rnd.Next(0, 3)],
textAnswer);
    }
}
```

Команда активируется при использовании слов привет, здравствуй и добрый день.

Выполнение команды заключается в выборе случайного перезаписанного ответа и возврата его названия для дальнейшего воспроизведения.

5.2 Команда «Запуск»

Класс OpenCommand (приложение Б) реализует интерфейс ICommand и предназначен для выполнения команды открытия различных программ на компьютере на основе голосовых запросов. Этот класс содержит логику для распознавания, конфигурирования и запуска программ.

Основные компоненты и функции:

1) Поля и свойства:

- TargetWords: массив строк, содержащий слова-триггеры для распознавания команды (например, "открой", "включи", "запусти").
- PathToLinks: строка, содержащая путь к конфигурационному файлу с программами и их путями запуска.
- links: словарь, сопоставляющий названия программ с их путями запуска.
- PathsToAnswer: массив строк с путями к аудиофайлам ответов голосового ассистента.
- commandParams: список строк, содержащий параметры команд, которые может распознать и обработать команда.

2) Конструктор:

- Инициализирует TargetWords и PathsToAnswer с предопределенными значениями.
- Инициализирует пустые коллекции для links и commandParams.

3) Методы:

- Execute: основной метод для выполнения команды. Принимает список слов запроса, конфигурирует параметры, распознает параметр команды и запускает соответствующую

программу. Возвращает кортеж, содержащий путь к аудиофайлу ответа и булево значение, указывающее на успешность выполнения команды.

- `GetParam`: извлекает параметр команды из списка слов запроса. Использует метод `Manager.GetCommandParam` для поиска совпадений.
- `LaunchProgramm`: запускает программу по её названию, если она найдена в словаре `links`. Использует класс `Process` для запуска программ.
- `Configure`: читает конфигурационный файл `JSON` и заполняет словарь `links` и список `commandParams` значениями из файла. Обработывает исключения, связанные с чтением и парсингом файла.

У данной команды есть свой конфигурационный файл, в котором хранятся названия программ, ссылок, сценариев и пути, ссылки, список программ в сценарии для их запуска.

5.3 Команда «Управление звуком»

5.3.1 Вспомогательный класс `AudioManager`

Код класса `AudioManager` находится в приложении В. Класс `AudioManager` предоставляет методы для управления громкостью и состоянием звука основного аудиоустройства системы.

Класс содержит два поля: `deviceEnumerator` для перечисления аудиоустройств и `defaultDevice`, которое хранит устройство воспроизведения по умолчанию. Конструктор класса инициализирует эти поля, создавая экземпляр `MMDeviceEnumerator` и получая аудиоустройство по умолчанию для воспроизведения мультимедиа.

Метод `GetVolume` возвращает текущий уровень громкости устройства в процентах, преобразуя значение из диапазона `[0.0, 1.0]` в диапазон `[0, 100]`.

Метод `SetVolume` устанавливает уровень громкости в процентах, преобразуя значение из диапазона `[0, 100]` обратно в диапазон `[0.0, 1.0]`.

Методы `IncreaseVolume` и `DecreaseVolume` соответственно увеличивают или уменьшают уровень громкости на заданное значение, проверяя, чтобы новое значение не превышало 100 и не было меньше 0.

Метод `IsMuted` возвращает текущее состояние звука (включен или выключен). Методы `Mute` и `Unmute` соответственно выключают и включают звук. Метод `ToggleMute` переключает состояние звука между "включен" и "выключен".

5.3.2 Основной класс `SoundControlCommand`

Код данного класса находится в приложении Г. Класс `SoundControlCommand` предназначен для управления звуком на компьютере на основе распознанных команд. Он реализует интерфейс `ICommand` и содержит логику для выполнения команды изменения уровня громкости или включения/выключения звука.

`TargetWords`: Массив строк, содержащий ключевые слова, которые могут быть использованы для активации данной команды. Например, "громкость" и "звук".

`PathsToAnswer`: Массив строк, содержащий пути к аудиофайлам, которые будут проигрываться в ответ на команду. Это ответы, которые можно воспроизвести пользователю.

`commandParams`: Список строк, содержащий параметры команд, такие как "максимум" и "минимум", для управления громкостью.

Конструктор инициализирует поля `TargetWords`, `PathsToAnswer` и `commandParams`. Это обеспечивает начальные значения для ключевых слов и параметров, используемых в командах.

Метод `Execute` выполняет основную логику команды. Он принимает список строк `request`, представляющих распознанные слова команды, и возвращает кортеж из строки и булевого значения. Строка представляет путь к

аудиофайлу с ответом, а булево значение указывает, нужно ли синтезировать ответ.

Логика выполнения:

- 1) Создается экземпляр AudioManager для управления громкостью.
- 2) Инициализируется случайный генератор (Random).
- 3) Перебираются слова из request для поиска параметра команды с помощью метода Manager.GetCommandParam.
- 4) Если параметр команды не найден, анализируются слова для поиска числового значения громкости.
- 5) Если числовое значение найдено, оно преобразуется в целое число и устанавливается как уровень громкости с помощью audioManager.SetVolume.
- 6) Если параметр команды равен "максимум", громкость устанавливается на 100%.
- 7) Если параметр команды равен "минимум", звук выключается (мутируется) с помощью audioManager.Mute.
- 8) Возвращается кортеж со случайно выбранным путем к аудиофайлу ответа и значением SynthesizeAnswer (в данном случае всегда false).

5.3.3 Вспомогательный класс StringToNumberConverter

Код класса находится в приложении Д. Класс StringToNumberConverter предназначен для преобразования числительных на русском языке в целочисленные значения. Он находится в пространстве имен CommandsManager.Commands.HelloCommand и включает два метода: Convert и IsNumberWord.

Класс содержит три словаря, которые сопоставляют строковые представления числительных с их числовыми значениями:

- units: содержит числительные от "ноль" до "девять".
- teens: содержит числительные от "десять" до "девятнадцать".
- tens: содержит числительные для десятков от "двадцать" до "девяносто".

Метод `Convert` принимает строку `numberWord`, представляющую числительное, и возвращает соответствующее целочисленное значение.

- 1) Метод приводит строку к нижнему регистру и убирает лишние пробелы.
- 2) Затем он проверяет, содержится ли строка в словаре `units`, и возвращает соответствующее значение, если содержится.
- 3) Если строка не найдена в `units`, проверяется словарь `teens`.
- 4) Если строка не найдена в `units` и `teens`, метод проверяет, начинается ли строка с одного из ключей в словаре `tens`.
- 5) Если строка совпадает с одним из ключей в `tens`, возвращается соответствующее значение.
- 6) Если строка начинается с ключа из `tens`, но содержит дополнительные символы, метод проверяет, содержит ли оставшаяся часть строки числительное из `units`, и если да, возвращает сумму значений.
- 7) Если строка не соответствует ни одному из перечисленных условий, метод возвращает `-1`, указывая на то, что преобразование не удалось.

Метод `IsNumberWord` проверяет, является ли данное слово числительным, возвращая `true` или `false`.

- 1) Метод приводит строку к нижнему регистру и убирает лишние пробелы.
- 2) Затем проверяет, содержится ли слово в любом из словарей: `units`, `teens` или `tens`.

5.4 Команда «Погода»

5.4.1 Вспомогательный класс `WeatherService`

Код класса находится в приложении Е. Класс `WeatherService` предназначен для получения текущей погоды для заданного города с использованием API `OpenWeatherMap`. Он предоставляет асинхронный метод `GetWeatherAsync` для выполнения запроса к API и возвращает строку с описанием погоды и температурой.

`_apiKey`: Закрытое поле, хранящее ключ API, используемый для аутентификации при запросе к OpenWeatherMap.

Конструктор `WeatherService(string apiKey)` инициализирует поле `_apiKey`.

Метод `GetWeatherAsync` выполняет следующие действия:

- 1) Извлекает название города из запроса с помощью метода `ExtractCityName`.
- 2) Если название города не может быть извлечено, возвращает `null`.
- 3) Формирует URL для запроса к OpenWeatherMap API, используя название города и ключ API.
- 4) Создает экземпляр `HttpClient` и выполняет HTTP GET запрос к сформированному URL.
- 5) Проверяет, успешно ли выполнен запрос. Если запрос неуспешен, возвращает `null`.
- 6) Читает и парсит ответ в формате JSON.
- 7) Извлекает описание погоды и температуру из полученного JSON-объекта.
- 8) Возвращает строку, содержащую информацию о погоде в заданном городе.

Метод `ExtractCityName` использует регулярное выражение для извлечения названия города из строки запроса. Регулярное выражение ищет слово, следующее за предлогами "в" или "во", и возвращает его. Если название города не найдено, возвращает пустую строку.

5.4.2 Основной класс WeatherCommand

Код класса находится в приложении Ж. Класс `WeatherCommand` предназначен для получения информации о погоде. Он реализует интерфейс `ICommand` и взаимодействует с классом `WeatherService` для получения данных о погоде.

TargetWords: Массив строк, содержащий ключевые слова, которые могут быть использованы для активации данной команды. В данном случае это слово "погода".

PathsToAnswer: Массив строк, содержащий пути к аудиофайлам, которые будут проигрываться в случае ошибки.

Конструктор инициализирует поля **TargetWords** и **PathsToAnswer** значениями по умолчанию.

Метод **Execute** выполняет основную логику команды. Он принимает список строк **commaandParams**, представляющих параметры команды, и возвращает кортеж из строки и булевого значения. Строка представляет результат выполнения команды, а булево значение указывает, следует ли синтезировать ответ.

Логика выполнения:

- 1) Инициализируется переменная **textAnswer** значением **false**.
- 2) Создается экземпляр **WeatherService** с заданным API-ключом.
- 3) Выполняется асинхронный запрос к методу **GetWeatherAsync** класса **WeatherService**, передавая параметры команды, объединенные в одну строку.
- 4) Метод **task.Wait()** ожидает завершения асинхронной операции.
- 5) Полученный результат сохраняется в переменную **result**.
- 6) Если результат равен **null** (что указывает на ошибку или отсутствие данных), возвращается кортеж с путем к аудиофайлу ошибки и значением **false**.
- 7) В противном случае возвращается кортеж с результатом выполнения команды и значением **true**.

5.5 Команда «Поиск в википедии»

Класс **WikipediaSearcher** (листинг 5.3) предназначен для выполнения поиска по Википедии и получения краткого описания заданного поискового запроса. Он использует HTTP-запросы для взаимодействия с API Википедии и возвращает текстовый результат.

Листинг 5.3 – Класс WikipediaSearcher

```
public class WikipediaSearcher
{
    private static readonly HttpClient client = new
HttpClient();
    public static async Task<string> GetWikipediaSummary(string
searchQuery)
    {
        string apiUrl =
$"https://ru.wikipedia.org/api/rest_v1/page/summary/{Uri.Escape
DataString(searchQuery)}";

        HttpResponseMessage response = await
client.GetAsync(apiUrl);

        if (response.IsSuccessStatusCode)
        {
            string jsonResponse = await
response.Content.ReadAsStringAsync();
            JObject json = JObject.Parse(jsonResponse);

            var result = json["extract"]?.ToString() ??
"Описание не найдено.";
            var test = result.Split('.');
            if (test.Length > 1)
            {
                return test[0];
            }
            return result;
        }
        else
        {
            return null;
        }
    }
}
```

Статическое поле `client`, содержит экземпляр `HttpClient`. Это позволяет использовать один и тот же клиент для всех запросов, улучшая производительность за счет повторного использования соединений.

Метод `GetWikipediaSummary` является асинхронным и выполняет следующие действия:

Формирование URL: формируется URL для запроса к API Википедии, используя `Uri.EscapeDataString` для корректного кодирования поискового запроса.

Выполнение HTTP-запроса: используется HttpClient для отправки GET-запроса к сформированному URL.

Обработка ответа:

- 1) Проверяет, успешен ли HTTP-ответ (response.IsSuccessStatusCode).
- 2) Если запрос выполнен успешно, читает содержимое ответа в строку (jsonResponse).
- 3) Парсит JSON-ответ с помощью JObject.Parse.
- 4) Извлекает значение поля extract из JSON-объекта, которое содержит краткое описание статьи.
- 5) Если поле extract отсутствует, возвращает строку "Описание не найдено."
- 6) Разделяет описание по точке и возвращает первую часть предложения, если оно состоит из нескольких предложений.
- 7) Если запрос не был успешен, возвращает null.

5.6 Алгоритм мягкого сравнения

Для поиска активационного слова и параметров команды используется алгоритм мягкого сравнения строк, основанный на алгоритме расстояния Левенштейна, что позволяет обнаружить ключевые слова, даже если они сказаны не совсем точно. Например, слова «привет» и «приветик» будут одинаково распознаны как ключевые слова приветственной команды.

В отличие от жесткого сравнения, которое требует точного соответствия строк, мягкое сравнение учитывает различные проблемы, такие как опечатки, вставки, удаления и перестановки символов.

Основная идея алгоритма мягкого сравнения заключается в вычислении коэффициента схожести между двумя строками на основе количества совпадающих элементов или подстрок. Результатом является число, обычно от 0 до 100, которое показывает, насколько строки схожи друг с другом.

6 РАЗРАБОТКА МОДУЛЯ «МЕНЕДЖЕР КОМАНД»

В проекте модуль менеджер команд представлен классом `Manager`. Полем класса является словарь. Ключом является активационное слово команды, представленное в виде строки. Значением является класс команды, один из тех, что был рассмотрен в разделе 5.

Конструктор класса представлен на листинге 6.1.

Листинг 6.1 – конструктор класса `Manager`

```
public Manager()
{
    var allTypes = Assembly.GetExecutingAssembly().GetTypes();

    var scriptClasses = allTypes.Where(t =>
        typeof(Commands.ICommand).IsAssignableFrom(t) &&
        !t.IsInterface);

    foreach (var scriptClass in scriptClasses)
    {
        CommandsManager.Commands.ICommand command =
            (Commands.ICommand)Activator.CreateInstance(scriptClass);

        foreach (var targetWord in command.TargetWords)
        {
            Commands[targetWord] = command;
        }
    }
}
```

Конструктор `Manager` выполняет динамическое обнаружение и инициализацию всех классов, реализующих интерфейс `ICommand`, из текущей сборки. Он добавляет экземпляры этих классов в словарь `Commands`, используя ключевые слова, ассоциированные с каждой командой.

Метод `GetCommand` (листинг 6.2.) предназначен для получения наиболее подходящей команды на основе заданного имени команды. Он использует алгоритм для оценки схожести строк, чтобы найти команду, ключевое слово которой похоже на предоставленное имя команды.

Пошаговое описание работы метода:

1) Инициализация переменных:

- `command`: Переменная для хранения наиболее подходящей команды, инициализируется значением `null`.
- `bestMatchScore`: Переменная для хранения наилучшего найденного значения схожести, инициализируется значением 0.
- `threshold`: Пороговое значение для схожести, ниже которого команда не будет считаться подходящей, установлено на 70.

2) Поиск наилучшего совпадения:

- Метод проходит через все пары ключ-значение (`kvp`) в словаре `Commands`.
- Для каждого ключевого слова вычисляется значение схожести (`similarityScore`) с помощью метода `Fuzz.Ratio(name, kvp.Key)`. Метод `Fuzz.Ratio` оценивает схожесть двух строк и возвращает числовое значение, где большее значение означает большую схожесть.
- Если значение схожести для текущей пары больше, чем текущее наилучшее значение (`bestMatchScore`), то обновляются `bestMatchScore` и `command`.

3) Проверка порога схожести:

- После прохождения всех элементов, если наилучшее значение схожести меньше порогового значения (`threshold`), устанавливается `command` в `null`.

4) Возвращение результата:

- Метод возвращает найденную команду, если значение схожести превышает пороговое значение, иначе возвращает null.

Листинг 6.2 – метод GetCommand

```
public Commands.ICommand GetCommand(string name)
{
    Commands.ICommand command = null;
    int bestMatchScore = 0;
    int threshold = 70;
    foreach (var kvp in Commands)
    {
        int similarityScore = Fuzz.Ratio(name, kvp.Key);
        if (similarityScore > bestMatchScore)
        {
            bestMatchScore = similarityScore;
            command = kvp.Value;
        }
    }
    if (bestMatchScore < threshold)
    {
        command = null;
    }
    return command;
}
```

Метод GetCommandParam (листинг 6.3) предназначен для нахождения параметра команды, который наиболее близко соответствует заданному, используя алгоритм оценки схожести строк. Метод возвращает наиболее подходящий параметр, если его схожесть превышает заданный порог.

Листинг 6.3 – Метод GetCommandParam

```
public static string GetCommandParam(string name, List<string>
commandParams)
{
    string result = null;

    int bestMatchScore = 0;

    int threshold = 50;

    foreach (var p in commandParams)
    {
        int similarityScore = Fuzz.Ratio(name, p);
        if (similarityScore > bestMatchScore)
        {
            bestMatchScore = similarityScore;
            result = p;
        }
    }
}
```

```

        }
    }
    if (bestMatchScore < threshold)
    {
        result = null;
    }

    return result;
}

```

Пошаговое описание работы метода:

1) Инициализация переменных:

- **result**: Переменная для хранения наиболее подходящего параметра, инициализируется значением `null`.
- **bestMatchScore**: Переменная для хранения наилучшего найденного значения схожести, инициализируется значением `0`.
- **threshold**: Пороговое значение для схожести, ниже которого параметр не будет считаться подходящим, установлено на `50`.

2) Поиск наилучшего совпадения:

- Метод проходит через все элементы списка `commandParams`.
- Для каждого параметра вычисляется значение схожести (`similarityScore`) с помощью метода `Fuzz.Ratio(name, p)`. Метод `Fuzz.Ratio` оценивает схожесть двух строк и возвращает числовое значение, где большее значение означает большую схожесть.
- Если значение схожести для текущего параметра больше, чем текущее наилучшее значение (`bestMatchScore`), то обновляются `bestMatchScore` и `result`.

3) Проверка порога схожести:

- После прохождения всех элементов, если наилучшее значение схожести меньше порогового значения (`threshold`), устанавливается `result` в `null`.

4) Возвращение результата:

- Метод возвращает найденный параметр, если значение схожести превышает пороговое значение, иначе возвращает `null`.

7 РАЗРАБОТКА МОДУЛЯ СИНТЕЗА РЕЧИ

Модуль синтеза речи представлен классом Synthesizer. Метод Synthesis (листинг 7.1) выполняет синтез речи на основе предоставленного текста с использованием API Yandex.TTS (Text-to-Speech). Метод асинхронно преобразует текст в аудиоформат MP3 и возвращает его в виде массива байт. Если во время выполнения запроса возникает ошибка, метод возвращает null.

Пошаговое описание работы метода:

- 1) Метод создает экземпляр клиента IYandexTts, используя класс YandexTtsClient и передавая ему конфигурацию YandexTtsConfig, в которую включен API-ключ и включен логгинг.
- 2) В блоке try метод отправляет запрос на синтез речи, используя метод TextToSpeechAsync клиента IYandexTts. В этот метод передается текст для синтеза, параметры голоса (в данном случае VoiceParameters.Ermil), и формат аудио (в данном случае AudioFormat.Mp3).
- 3) Если запрос успешно выполнен, метод возвращает содержимое результата (result.Content) в виде массива байт.
- 4) Если возникает исключение YandexTtsServiceException (указывающее на ошибку службы Яндекс.ТТС), метод ловит его в блоке catch и возвращает null.

Листинг 7.1 – метод Synthesis

```
public async Task<byte[]> Synthesis(string textToSynthesis)
{
    IYandexTts client = new YandexTtsClient(new
    YandexTtsConfig(apiKey) { LoggingEnabled = true });
    try
    {
        var result = await client
            .TextToSpeechAsync(textToSynthesis,
                VoiceParameters.Ermil, AudioFormat.Mp3);
        return result.Content;
    }
    catch (YandexTtsServiceException)
    {
        return null;
    }
}
```


Метод `PlaySynthesisTextAsync` (листинг 7.2) имеет 2 перегрузки. Первая принимает аудио буфер, содержащий синтезированный текст в формате MP3, и асинхронно воспроизводит его. Если переданный буфер не равен `null`, метод сначала сохраняет аудио буфер в файл `"example.mp3"`. Затем он использует поток памяти (`MemoryStream`) для чтения MP3 данных и создает объект `Mp3FileReader`, который декодирует MP3 аудио данные для воспроизведения. Далее, метод создает объект `WaveOutEvent` для воспроизведения аудио. Аудио воспроизводится асинхронно в отдельной задаче с помощью метода `Task.Run`. Метод завершает работу, когда состояние воспроизведения изменяется с `Playing` на другое значение, что означает завершение воспроизведения.

Листинг 7.2 метод `PlaySynthesisTextAsync`

```
public async Task PlaySynthesisTextAsync(byte[] audioBuffer)
{
    if (audioBuffer != null)
    {
        using (FileStream fileStream = new
FileStream("example.mp3", FileMode.Create, FileAccess.Write))
        {
            await fileStream.WriteAsync(audioBuffer, 0,
audioBuffer.Length);
        }

        using (MemoryStream mp3Stream = new
MemoryStream(audioBuffer))
        {
            using (Mp3FileReader mp3Reader = new
Mp3FileReader(mp3Stream))
            {
                using (WaveOutEvent waveOut = new
WaveOutEvent())
                {
                    waveOut.Init(mp3Reader);

                    await Task.Run(() => waveOut.Play());

                    while (waveOut.PlaybackState ==
PlaybackState.Playing)
                    {
                    }
                }
            }
        }
    }
}
```

Вторая перегрузка метода `PlaySynthesisTextAsync` (листинг 7.3) асинхронно воспроизводит аудиофайл по указанному пути. Сначала он открывает файл и считывает его содержимое в массив байт `audioBuffer`. Если массив не равен `null`, метод создает поток памяти (`MemoryStream`) с этими данными и использует `Mp3FileReader` для декодирования MP3 аудио данных. Затем создается объект `WaveOutEvent` для воспроизведения аудио.

Для асинхронного ожидания завершения воспроизведения используется объект `TaskCompletionSource<bool>`. Обработчик события `PlaybackStopped` указывает, когда воспроизведение завершилось, вызывая метод `SetResult` у `TaskCompletionSource`. Это позволяет методу асинхронно ожидать завершения воспроизведения, используя `await tcs.Task`.

Листинг 7.3 – Вторая перегрузка метода `PlaySynthesisTextAsync`

```
public async Task PlaySynthesisTextAsync(string pathToAnswer)
{
    byte[] audioBuffer;

    using (FileStream fs = new
FileStream(pathToRecordedResponse + pathToAnswer,
FileMode.Open, FileAccess.Read))
    {
        audioBuffer = new byte[fs.Length];
        await fs.ReadAsync(audioBuffer, 0, (int)fs.Length);
    }

    if (audioBuffer != null)
    {
        using (MemoryStream mp3Stream = new
MemoryStream(audioBuffer))
        {
            using (Mp3FileReader mp3Reader = new
Mp3FileReader(mp3Stream))
            {
                using (WaveOutEvent waveOut = new
WaveOutEvent())
                {
                    var tcs = new TaskCompletionSource<bool>();

                    waveOut.PlaybackStopped += (sender, args)
=>
                    {
                        tcs.SetResult(true);
                    }
                }
            }
        }
    }
}
```

```
};  
  
waveOut.Init(mp3Reader);  
  
waveOut.Play();  
  
await tcs.Task;  
}  
}  
}  
}  
}
```

ЗАКЛЮЧЕНИЕ

В ходе работы над дипломной работой был разработан функциональный голосовой ассистент для ОС Windows, который сможет обрабатывать и выполнять команды пользователя, используя технологии распознавания и синтеза речи. Выполнены следующие задачи:

- 1) Были исследованы и проанализированы существующие решения:
 - Проведен анализ существующих голосовых ассистентов и их функционала;
 - Проведено изучение технологий и библиотек для распознавания и синтеза речи, доступных для платформы Windows.
- 2) Разработана архитектура программы:
 - Определены основные компоненты системы;
 - Разработана схема взаимодействия между компонентами.
- 3) Реализован функционал голосового управления:
 - Интегрирована технология распознавания речи для обработки голосовых команд;
 - Разработан модуль «Менеджер команд» для обработки естественного языка;
 - Реализован функционал синтеза речи для обратной связи с пользователем.

Были получены навыки:

- Проектирования архитектуры программы;
- Работа с языками программирования C# и Python;
- Работа с инструментами Kaldi, VOSK, Picovoice, Yandex SpeechKit, NAudio.

В дальнейшем планируется:

- Расширение списка поддерживаемых команд;
- Внедрение системы управления умным домом;
- Доработка и оптимизация пользовательского интерфейса;
- Полноценное решение проблемы API ключей;
- Реализация способа взаимодействия с голосовым ассистентом через текстовый чат.

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В пояснительной записке применяются следующие сокращения и обозначения:

TTS – text to speech (Преобразование текста в речь).

NLP – natural language processing (Обработка естественного языка).

STT – speech to text (Преобразование речи в текст).

API – Application Programming Interface (Программный интерфейс приложения)

ОС – Операционная система.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 ГОСТ 2.32–2017 СИБИД. Отчёт о научно-исследовательской работе. Структура и правила оформления (с поправками) от 24 октября 2017 г. – М. 2017 г. URL: <https://docs.cntd.ru/document/1200157208> (дата обращения 202-16-02)
- 2 Блог компании Яндекс. Извлечение объектов и фактов из текстов в Яндексе. Лекция для Малого ШАДа // habr : [сайт]. — URL: <https://habr.com/ru/companies/yandex/articles/205198/> (дата обращения: 01.05.2024).
- 3 Обработка естественного языка // Документация яндекс: [сайт]. — URL: <https://yandex.ru/dev/dialogs/alice/doc/nlu.html> (дата обращения: 01.05.2024).
- 4 Porcupine Wake Word // Picovoice documentation: [сайт]. — URL: <https://picovoice.ai/docs/porcupine/> (дата обращения: 05.05.2024).
- 5 Yandex SpeechKit // Yandex cloud: [сайт]. — URL: <https://yandex.cloud/ru/docs/speechkit/> (дата обращения: 03.05.2024).
- 6 Integrations // Alpha Cephei: [сайт]. — URL: <https://alphacephei.com/vosk/> (дата обращения: 10.05.2024).
- 7 Коэн Майкл. "Дизайн голосового пользовательского интерфейса". Издательство: Наука и Техника, 2020. 272 с.
- 8 Справочник по языку C#// learn.microsoft : [сайт]. — URL: <https://learn.microsoft.com/ru-ru/dotnet/csharp/language-reference/> (дата обращения: 29.04.2024).
- 9 Беленко М. В. Балакшин П.В. СРАВНИТЕЛЬНЫЙ АНАЛИЗ СИСТЕМ РАСПОЗНАВАНИЯ РЕЧИ С ОТКРЫТЫМ КОДОМ / Беленко М.В. Балакшин П.В. // research-journal.org: [сайт]. — URL: <https://research-journal.org/archive/4-58-2017-april/sravnitelnyj-analiz-sistem-raspoznavaniya-rechi-s-otkrytym-kodom> (дата обращения: 25.04.2024).

10 Расстояние Левенштейна для чайников / [Электронный ресурс] // habr.com: [сайт]. — URL: <https://habr.com/ru/articles/676858/> (дата обращения: 15.05.2024).

ПРИЛОЖЕНИЕ А

Класс Recognizer

Листинг А.1. – Функция RecognizeAsync

```
public async Task<string> RecognizeAsync()
{
    await Task.Delay(TimeSpan.FromSeconds(0.1));
    string result = "{\"text\" : \"\"}";

    using (var waveSource = new WaveInEvent())
    {
        waveSource.WaveFormat = new WaveFormat(SampleRate, 1);

        var taskCompletionSource = new
TaskCompletionSource<bool>();

        waveSource.DataAvailable += (sender, e) =>
        {
            if (rec.AcceptWaveform(e.Buffer, e.BytesRecorded))
            {
                result = rec.FinalResult();
                waveSource.StopRecording();
            }
        };

        waveSource.RecordingStopped += (sender, e) =>
taskCompletionSource.SetResult(true); // Устанавливаем
результат завершения записи

        waveSource.StartRecording();

        // Ожидаем завершения записи асинхронно
        await taskCompletionSource.Task;

        return JObject.Parse(result) ["text"].ToString();
    }
}
```

Листинг А.2. – Функция WakeWordRecognitionAsync

```
public async Task<bool> WakeWordRecognitionAsync()
{
    using (PvRecorder recorder =
PvRecorder.Create(frameLength: porcupine.FrameLength))
    {
        recorder.Start();
        while (recorder.IsRecording)
        {
            Task<short[]> readTask = Task.Run(() =>
recorder.Read());
            short[] frame = await readTask;
            int result = porcupine.Process(frame);
            if (result >= 0)
            {
                recorder.Stop();
                return true;
            }
        }
        return false;
    }
}
```

ПРИЛОЖЕНИЕ Б

Класс OpenCommand

```
public class OpenCommand : ICommand
{
    public string[] TargetWords { get; private set; }
    private static readonly string PathToLinks =
"C:\\Users\\defen\\source\\repos\\VoiceAssistant\\CommandsManager\\Commands\\OpenCommand\\OpenCommandConfig.json";
    private Dictionary<string, string> links = [];
    public string[] PathsToAnswer { get; private set; }
    private List<string> commandParams = [];

    public OpenCommand()
    {
        TargetWords = ["открой", "включи", "запусти"];
        PathsToAnswer = ["OpenV1.mp3", "OpenV2.mp3",
"OpenV3.mp3", "OpenExceptionV1.mp3"];
    }

    public Tuple<string, bool> Execute(List<string> request)
    {
        var isConfigure = Configure();

        if (!isConfigure)
            return new Tuple<string, bool>(PathsToAnswer[3],
false);

        bool SynthesizeAnswer = false;
        var rnd = new Random();
        string resultParam = GetParam(request);

        if (resultParam == null)
        {
            return new Tuple<string, bool>(PathsToAnswer[3],
SynthesizeAnswer);
        }

        if (resultParam.Contains("режим"))
        {
            string[] programmes = links[resultParam].Split("
");
            foreach (string program in programmes)
            {
                bool IsLaunching = LaunchProgramm(program);

                if (!IsLaunching)
                    return new Tuple<string,
bool>(PathsToAnswer[3], false);
            }
        }
    }
}
```

```

        return new Tuple<string,
bool>(PathsToAnswer[rnd.Next(0, 3)], SynthesizeAnswer);
    }
    bool isLaunching = LaunchProgramm(resultParam);
    if (!isLaunching)
        return new Tuple<string, bool>(PathsToAnswer[3],
false);
    return new Tuple<string,
bool>(PathsToAnswer[rnd.Next(0, 3)], SynthesizeAnswer);
}
private string GetParam(List<string> request)
{
    string resultParam = null;
    var complexParametr = new StringBuilder();
    int countWordInComplexParametr = 0;

    foreach (string arg in request)
    {
        if (complexParametr.Length > 0)
        {
            complexParametr.Append(" ");
        }
        complexParametr.Append(arg);
        countWordInComplexParametr++;

        resultParam = Manager.GetCommandParam(arg,
commandParams);

        if (countWordInComplexParametr > 1)
        {
            resultParam =
Manager.GetCommandParam(complexParametr.ToString(),
commandParams);
        }
        if (resultParam != null)
        {
            return resultParam;
        }
    }
    return null;
}

private bool LaunchProgramm(string programmName)
{
    if (links.ContainsKey(programmName))
    {
        try
        {
            Process.Start(new ProcessStartInfo
            {

```

```

        FileName = links[programmName],
        UseShellExecute = true
    });

    }
    catch (Exception)
    {
        return false;
        //return new Tuple<string,
bool>(PathsToAnswer[3], false);
    }

    return true;

}

return false;

}

private bool Configure()
{
    try
    {
        if (!File.Exists(PathToLinks))
        {
            throw new
FileNotFoundException("Configuration file not found.");
        }

        string json = File.ReadAllText(PathToLinks);
        links =
JsonConvert.DeserializeObject<Dictionary<string,
string>>(json);

        commandParams.Clear();
        commandParams.AddRange(links.Keys);
    }
    catch (Exception)
    {
        return false;
    }

    return true;

}

}

}

```

ПРИЛОЖЕНИЕ В

Класс AudioManager

```
public class AudioManager
{
    private readonly MMDeviceEnumerator deviceEnumerator;
    private readonly MMDevice defaultDevice;

    public AudioManager()
    {
        deviceEnumerator = new MMDeviceEnumerator();
        defaultDevice =
deviceEnumerator.GetDefaultAudioEndpoint(DataFlow.Render,
Role.Multimedia);
    }

    // Получить текущий уровень громкости (в процентах)
    public int GetVolume()
    {
        return
(int)(defaultDevice.AudioEndpointVolume.MasterVolumeLevelScalar
* 100);
    }

    // Установить уровень громкости (в процентах)
    public void SetVolume(int volume)
    {
        float volumeScalar = volume / 100.0f;

defaultDevice.AudioEndpointVolume.MasterVolumeLevelScalar =
volumeScalar;
    }

    // Увеличить громкость на заданное значение
    public void IncreaseVolume(int increment)
    {
        int newVolume = GetVolume() + increment;
        if (newVolume > 100) newVolume = 100;
        SetVolume(newVolume);
    }

    // Уменьшить громкость на заданное значение
    public void DecreaseVolume(int decrement)
    {
        int newVolume = GetVolume() - decrement;
        if (newVolume < 0) newVolume = 0;
        SetVolume(newVolume);
    }

    // Проверить, включен ли звук
    public bool IsMuted()
```

```

    {
        return defaultDevice.AudioEndpointVolume.Mute;
    }

    // Включить звук
    public void Mute()
    {
        defaultDevice.AudioEndpointVolume.Mute = true;
    }

    // Отключить звук
    public void Unmute()
    {
        defaultDevice.AudioEndpointVolume.Mute = false;
    }

    // Переключить состояние звука (включить/отключить)
    public void ToggleMute()
    {
        defaultDevice.AudioEndpointVolume.Mute =
!defaultDevice.AudioEndpointVolume.Mute;
    }
}
}

```

ПРИЛОЖЕНИЕ Г

Класс SoundControlCommand

```
internal class SoundControlCommand: ICommand
{
    public string[] TargetWords { get; private set; }
    public string[] PathsToAnswer { get; private set; }
    private List<string> commandParams;

    public SoundControlCommand()
    {
        TargetWords = ["громкость", "звук",];
        PathsToAnswer = ["OpenV1.mp3", "OpenV2.mp3",
"OpenV3.mp3", "SoundExeptionV1.mp3"];
        commandParams = ["максимум", "минимум"];
    }

    public Tuple<string, bool> Execute(List<string> request)
    {
        AudioManager audioManager = new AudioManager();
        bool SynthesizeAnswer = false;
        var rnd = new Random();
        string resultParam = null;

        foreach (string arg in request)
        {
            resultParam = Manager.GetCommandParam(arg,
commandParams);

            if (resultParam != null)
            {
                break;
            }
        }

        string volumeNumber = "";

        int result = -1;

        if (resultParam == null)
        {
            foreach (string arg in request)
            {
                if (StringToNumberConverter.IsNumberWord(arg))
                {
                    volumeNumber += arg + " ";
                }
            }
        }
    }
}
```



```

        }

        result =
StringToNumberConverter.Convert(volumeNumber);

        if (result == -1)
        {
            return new Tuple<string,
bool>(PathsToAnswer[3], SynthesizeAnswer);
        }
        else
        {
            audioManager.SetVolume(result);
        }

        if(resultParam == commandParams[0])
        {
            audioManager.SetVolume(100);
        }

        if (resultParam == commandParams[1])
        {
            audioManager.Mute();
        }

        return new Tuple<string,
bool>(PathsToAnswer[rnd.Next(0, 3)], SynthesizeAnswer);
    }
}

```

ПРИЛОЖЕНИЕ Д

Класс StringToNumberConverter

```
public static class StringToNumberConverter
{
    private static readonly Dictionary<string, int> units =
new Dictionary<string, int>
{
    {"ноль", 0},
    {"один", 1},
    {"два", 2},
    {"три", 3},
    {"четыре", 4},
    {"пять", 5},
    {"шесть", 6},
    {"семь", 7},
    {"восемь", 8},
    {"девять", 9}
};

    private static readonly Dictionary<string, int> teens =
new Dictionary<string, int>
{
    {"десять", 10},
    {"одиннадцать", 11},
    {"двенадцать", 12},
    {"тринадцать", 13},
    {"четырнадцать", 14},
    {"пятнадцать", 15},
    {"шестнадцать", 16},
    {"семнадцать", 17},
    {"восемнадцать", 18},
    {"девятнадцать", 19}
};

    private static readonly Dictionary<string, int> tens =
new Dictionary<string, int>
{
    {"двадцать", 20},
    {"тридцать", 30},
    {"сорок", 40},
    {"пятьдесят", 50},
    {"шестьдесят", 60},
    {"семьдесят", 70},
    {"восемьдесят", 80},
    {"девяносто", 90}
};

    public static int Convert(string numberWord)
    {
        numberWord = numberWord.ToLower().Trim();
```

```

        if (units.ContainsKey(numberWord))
        {
            return units[numberWord];
        }
        if (teens.ContainsKey(numberWord))
        {
            return teens[numberWord];
        }
        foreach (var ten in tens)
        {
            if (numberWord.StartsWith(ten.Key))
            {
                if (numberWord == ten.Key)
                {
                    return ten.Value;
                }
                string unitPart =
numberWord.Substring(ten.Key.Length).Trim();
                if (units.ContainsKey(unitPart))
                {
                    return ten.Value + units[unitPart];
                }
            }
        }
        return -1;
    }
    public static bool IsNumberWord(string word)
    {
        word = word.ToLower().Trim();
        return units.ContainsKey(word) ||
            teens.ContainsKey(word) ||
            tens.ContainsKey(word);
    }
}

```

ПРИЛОЖЕНИЕ Е

Класс GetWeatherAsync

```
public async Task<string> GetWeatherAsync(string query)
{
    // Извлекаем название города из запроса
    string cityName = ExtractCityName(query);

    if (string.IsNullOrEmpty(cityName))
    {
        return null;
    }

    // Формируем URL для запроса к OpenWeatherMap API
    string url =
    $"http://api.openweathermap.org/data/2.5/weather?q={cityName}&a
    ppid={_apiKey}&units=metric&lang=ru";

    using (HttpClient client = new HttpClient())
    {
        try
        {
            HttpResponseMessage response = await
            client.GetAsync(url).ConfigureAwait(false);

            if (!response.IsSuccessStatusCode)
            {
                return null;
            }

            string responseData = await
            response.Content.ReadAsStringAsync();
            JObject weatherData =
            JObject.Parse(responseData);

            // Извлекаем нужную информацию из полученного
            JSON
            string description =
            weatherData["weather"][0]["description"].ToString();
            string temperature =
            weatherData["main"]["temp"].ToString();

            return $"Погода в {cityName}: {description},
            температура: {temperature} градусов";
        }
        catch (Exception ex)
        {
            return null;
        }
    }
}
```

```

private string ExtractCityName(string query)
{
    // Используем регулярное выражение для поиска названия
города после "в"
    var match = Regex.Match(query, @"(?:в|во)\s+([А-Яа-яёЁа-
zА-З\с]+)", RegexOptions.IgnoreCase);
    return match.Success ? match.Groups[1].Value.Trim() :
string.Empty;
}
}

```

ПРИЛОЖЕНИЕ Ж

Класс WeatherCommand

```
public class WeatherCommand: ICommand
{
    public string[] TargetWords { get; private set; }
    public string[] PathsToAnswer { get; private set; }
    public WeatherCommand()
    {
        TargetWords = ["порода"];
        PathsToAnswer = ["WeatherException.mp3"];
    }

    public Tuple<string, bool> Execute(List<string>
commaandParams)
    {
        bool textAnswer = false;

        WeatherService weatherService = new
WeatherService("0eaf24888afb7eeal514593c058edff2");

        Task<string> task =
weatherService.GetWeatherAsync(string.Join("
commaandParams));

        task.Wait();

        string result = task.Result;

        if (result == null)
        {

            return new Tuple<string, bool>(PathsToAnswer[0],
textAnswer);
        }
        else
            return new Tuple<string, bool>(result, true);
    }
}
```